

Modularity Maximization: An Experimental Survey

Adil Chhabra
adil.chhabra@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Kenneth Langedal*
kenneth.langedal@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Christian Schulz
christian.schulz@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Abstract

There has been a wide variety of community detection methods proposed over the last two decades that optimize the modularity objective. Despite this abundance of methods, comparative experimental evaluations remain fragmented: most studies consider only a small subset of available algorithms. As a result, it is often unclear how different modularity-based methods relate to one another in practice, or under which conditions specific design choices are most effective. Most notably, the body of work can largely be divided in two main categories based on whether they use machine learning or not.

In this work, we present an experimental survey of modularity optimization methods, covering more than twenty representative approaches spanning different algorithmic paradigms. We summarize key algorithmic features, and provide a systematic experimental comparison across multiple datasets and evaluation criteria. Our goal is not to identify a single “best” method, but to clarify empirical trade-offs, expose gaps in existing comparisons, and offer practical guidance for researchers and practitioners working with modularity-based graph clustering.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Mathematics of computing** → **Graph theory**; • **Computing methodologies** → **Cluster analysis**.

Keywords

Community detection, Graph clustering, Modularity optimization, Experimental evaluation

ACM Reference Format:

Adil Chhabra, Kenneth Langedal, and Christian Schulz. 2018. Modularity Maximization: An Experimental Survey. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXX.XXXXXXX>

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXX.XXXXXXX>

1 Introduction

Community detection in graphs.

- Problem definition
- Why modularity?
- State why this survey is needed, missing comparisons in the literature
- At the end, list of contributions (experimental survey stuff)

2 Preliminary

We use the term algorithm to denote a concrete computational procedure. Except where explicitly stated, the algorithms considered here are heuristics for modularity maximization and do not guarantee optimality.

3 Related Work

4 Methods

This section contains a short description outlining the core ideas behind each clustering method. They are grouped the algorithms into categories to make it easier to navigate the survey. With in each category, the algorithms are presented in order of publication date. At the top level of this taxonomy, we have *Learning-based* methods and *Classical modularity optimization* methods. For this distinction, *Learning-based* refers to any method that train model parameters as part of the algorithm.

4.1 Classical Modularity Optimization Methods

CNM. A scalable greedy modularity-optimization algorithm with near-linear running time on large sparse networks and demonstrates its effectiveness on a large real-world network.

Walktrap. Based on the idea that random walks on a graph tend to get “trapped” into densely connected parts corresponding to communities.

Infomap. Infomap frames community detection as an information compression problem: it partitions a graph so that the expected code length of a random walk on the network is minimized, thus uncovering modules where flows concentrate. The alternative *Map Equation* objective function was also introduced here.

WT. TODO.

Louvain. The famous Louvain method, named after the location the work took place, is a multi-level approach to graph clustering. At each level, the algorithm greedily moves vertices to neighboring clusters based on the modularity delta. Once there are no improving moves, each cluster is contracted into a single vertex, and the process repeats.

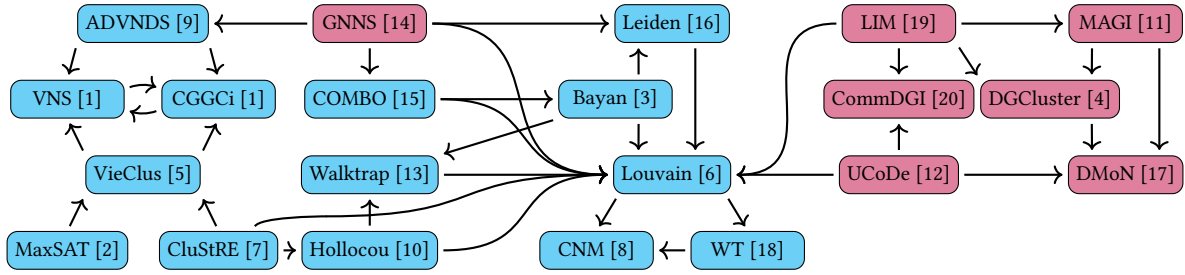


Figure 1: Previous experimental comparisons visualized. Each directed edge indicates that the source algorithm used the destination algorithm in the experimental evaluation. The colors indicate either a learning-based method when red, or a classical optimization algorithm when blue.

Table 1: Comparison of the methods included in the experimental survey. The *Code Available* field is defined as follows: × means not available, one ★ means code is available but not prepared for use in any way, ★★ is for methods with open source implementations with well defined input/output formats, and ★★★ means the algorithm is easy to use via portable Python packages such as the CDlib package.

Method	Year	Code Available	Stochastic	Parallel	Learning-Based	Features	Fixed C	Exact	Approx.	Running Time
CNM	2004	★★★								$O(n \log^2 n)$
Walktrap	2005	★★★	✓							$O(n^2 \log n)$
WT	2007	★★★								Improved CNM
Louvain	2008	★★★	✓							?
CGGCI	2012	×	?							?
VNS	2012	×	✓							?
COMBO	2014	★★★	✓							?
ADVNDs	2017	×	✓							Search-based
Hollocou	2017	★★	?							?
VieClus	2018	★★	✓							?
Leiden	2019	★★★	✓	✓						?
DMoN	2020	★	✓	✓	✓	✓	✓			?
CommDGI	2020	★	✓	✓	✓	✓	✓			?
GNNs	2022	★	✓	✓	✓	✓				?
UCoDe	2023	★	✓	✓	✓	✓	✓			?
DGCluster	2023	★	✓	✓	✓	✓				?
MAGI	2024	★	✓	✓	✓	✓	✓			?
Bayan	2024	★★						✓		?
CluStRE	2025	★★	?							?
LIM	2025	★	✓	✓	✓	✓				?
MaxSAT	2025	?						✓		?

CGGCI-RG. Combination of the CGGC (Core Groups Graph Clustering) ensemble scheme with an underlying Randomized Greedy (RG) clustering algorithm. It was the best performing algorithm submitted to the 10th DIMACE Implementation Challenge.

VNS. Variable Neighborhood Search metaheuristic that obtained the second place in the 10th DIMACS Implementation Challenge. It was a close call between VNS and CGGCI-RG.

COMBO. TODO.

Hollocou. Modularity optimization in the streaming setting. For each streamed edge, Hollocou employs a simple decision strategy

based on a given parameter and the volumes of the communities of the two nodes.

VieClus. A general memetic algorithm that can be adapted to optimize different objective functions.

Leiden. As a direct improvement to the Louvain method, Leiden addresses a defect in the Louvain method that can lead to disconnected communities.

Bayan. An exact algorithm for modularity optimization that uses Integer Linear Programming solvers.

CluStRE. Improved streaming algorithm with both re-streaming and combined use of VieCLus in-memory after streaming and contraction.

MaxSAT. An improved exact method for modularity optimization using a MaxSAT formulation.

4.2 Learning-Based Methods

Methods that find model parameters.

DMoN.

CommDGL.

UCoDe.

DGCluster.

MAGI.

5 Evaluation Methodology

In the following section, we present the evaluation methodology used in this experimental survey. The main contribution is the adaptation of 18 clustering algorithms behind one unified interface. In addition to enabling the experiments presented in this survey, it will also serve as an openly available resource for future work on graph clustering, simplifying the evaluation of older methods on new data or machines. This section also includes details on the datasets we used to evaluate the clustering algorithms and the computational environment.

5.1 Implementation

As shown in Table 1, over half of the clustering algorithms in the literature lack ready-to-use interfaces for clustering new data. Furthermore, among those with properly documented code, there are wide differences in the input graph format, output formats, and the additional information they report, such as time and memory consumption. As a result, it can take significant effort to get a particular algorithm to run and extract the results.

To address this problem, we aim to provide an interface with a single common input and output format and consistent time and memory monitoring. The interface exposes a common set of inputs, while allowing solver-specific parameters to be provided when applicable.

$$\text{CLUSTER}(G, T_{\max}, p, k, M_{\max}, \mathcal{F})$$

Where G is the input graph, $T_{\max}$ is the wall-clock time limit, p is the number of threads available to the algorithm, k is the number of clusters to compute, $M_{\max}$ is the memory limit in gigabytes, and $\mathcal{F}$ are the graph features.

Parameters p , k , and $\mathcal{F}$ are only used by a subset of the solvers. From Table 1, parallel solvers use p , those that compute a fixed number of clusters use k , and any solver that uses graph features can optionally accept $\mathcal{F}$. When $\mathcal{F}$ is not provided, but the solver expects features, a single 1.0 value is assigned to each vertex as the feature.

Table 2: Real-world instances with known ground truth clusters. The $|C|$ columns indicates the number of clusters.

Instance	n	m	$ \mathcal{F} $	$ C $
Amazon-Computers	13 752	245 861	767	10
Amazon-Photo	7 650	119 081	745	8
Citeseer	3 327	4 552	3 703	6
Coauthor-CS	18 333	81 894	6 805	15
Coauthor-Physics	34 493	247 962	8 415	5
Cora	2 708	5 278	1 433	7
ogbn-Arxiv	169 343	1 157 799	128	40
ogbn-Products	2 449 029	61 859 012	100	47
Pubmed	19 717	44 324	500	3
Reddit	232 965	57 307 946	602	41

Regarding code changes, we have tried to make as few modifications to the original source code as possible. Whenever possible, we use the authors' implementations or the most commonly used library implementation. The exact setup for each algorithm is described in our openly available GitHub repository¹. We did, however, make some deliberate changes to ensure consistent timing and quality measures.

- Parsing the input and writing the output to a file is not counted in the timing of the algorithm.
- For search-based algorithms that always run for the entire $T_{\max}$ duration, the recorded computation time is defined as the point in time when the best solution was found.
- Learning-based methods that train a neural network as part of the algorithm will break the training loop at $T_{\max}$, and use the current model at that time to produce the final clustering.

6 Dataset

We consider three datasets commonly used in the literature. The first set consists of real-world instances with known ground truth clusters. These graphs are commonly used by the machine learning algorithms. Next, we consider the instances used as part of the 10th DIMACS implementation challenge on graph clustering. This dataset is popular among the classical modularity optimization methods. Finally, we use a dataset of syntetically generated graph with node features. The particular generate we use allow us to explore the algorithms sensitivity to noise in both the graph and features. (TODO, rewrite this section with more details more details).

7 Experiments

8 Conclusion

¹<https://github.com/KennethLangedal/Clustering>

Table 3: Graph from the 10th DIMACS implementation challenge. There are no features or ground truth clusters for these graphs.

Instance	n	m
jazz	198	2 742
celegans-metabolic	453	2 025
email	1 133	5 451
PGPgiantcompo	10 680	24 316
adjnoun	112	425
as-22july06	22 963	48 436
astro-ph	16 706	121 251
celegansneural	297	2 148
cond-mat	16 726	47 594
cond-mat-2003	31 163	120 029
cond-mat-2005	40 421	175 691
dolphins	62	159
football	115	613
hep-th	8 361	15 751
karate	34	78
lesmis	77	254
netscience	1 589	2 742
polblogs	1 490	16 715
polbooks	105	441
power	4 941	6 594
cnr-2000	325 557	2 738 969
eu-2005	862 664	16 138 468
in-2004	1 382 908	13 591 473
uk-2002	18 520 486	261 787 258
uk-2007-05	105 896 555	3 301 876 564
road-central	14 081 816	16 933 413
road-usa	23 947 347	28 854 312
chesapeake	39	170
caidaRouterLevel	192 244	609 066
preferentialAttachment	100 000	499 985
smallworld	100 000	499 998
G-n-pin-pout	100 000	501 198

Acknowledgments

To Robert, for the bagels and explaining CMYK and color spaces.

References

- [1] Daniel Aloise, Gilles Caporossi, Pierre Hansen, Leo Liberti, Sylvain Perron, and Manuel Ruiz. 2012. Modularity maximization in networks by variable neighborhood search. In *Graph Partitioning and Graph Clustering*. Contemporary Mathematics, Vol. 588. American Mathematical Society, 113–127. doi:10.1090/conm/588/11705
- [2] Carlos Ansótegui, Vaidyanathan Peruvemba Ramaswamy, Stefan Szeider, and Hai Xia. 2025. Uncovering and Verifying Optimal Community Structure in Complex Networks: A MaxSAT Approach. In *Proceedings of the 25th International Conference on Computational Science (ICCS 2025)*. Springer Nature Switzerland, 35–49. doi:10.1007/978-3-031-97629-2_3
- [3] Samin Aref, Mahdi Mostajabdaveh, and Hriday Chheda. 2024. Bayan algorithm: Detecting communities in networks through exact and approximate optimization of modularity. *Physical Review E* 110, 4 (2024), 044315. doi:10.1103/PhysRevE.110.044315
- [4] Aritra Bhowmick, Mert Kosan, Zexi Huang, Ambuj Singh, and Sourav Medya. 2024. DGCLUSTER: A Neural Framework for Attributed Graph Clustering via Modularity Maximization. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*. AAAI Press, Washington, DC, USA, 11069–11077. doi:10.1609/aaai.v38i10.28983
- [5] Sonja Biedermann, Monika Henzinger, Christian Schulz, and Bernhard Schuster. 2018. Memetic Graph Clustering. In *Proceedings of the 17th International Symposium on Experimental Algorithms (SEA 2018)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 3:1–3:15. doi:10.4230/LIPIcs.SEA.2018.3
- [6] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008. doi:10.1088/1742-5468/2008/10/P10008
- [7] Adil Chhabra, Shai Dorian Peretz, and Christian Schulz. 2025. CluStRE: Streaming Graph Clustering with Multi-Stage Refinement. In *Proceedings of the 23rd International Symposium on Experimental Algorithms (SEA 2025)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 11:1–11:20. doi:10.4230/LIPIcs.SEA.2025.11
- [8] Aaron Clauset, Mark E. J. Newman, and Christopher Moore. 2004. Finding community structure in very large networks. *Physical Review E* 70, 6 (2004), 066111. doi:10.1103/PhysRevE.70.066111
- [9] Dušan Džamić, Daniel Aloise, and Nenad Mladenović. 2019. Ascent-descent variable neighborhood decomposition search for community detection by modularity maximization. *Annals of Operations Research* 272, 1 (2019). doi:10.1007/s10479-017-2553-9
- [10] Alexandre Holloco, Julien Maudet, Thomas Bonald, and Marc Lelarge. 2017. A Streaming Algorithm for Graph Clustering. arXiv preprint. doi:10.48550/arXiv.1712.04337
- [11] Yunfei Liu, Jintang Li, Yuehe Chen, Ruofan Wu, Ericbk Wang, Jing Zhou, Sheng Tian, Shuheng Shen, Xing Fu, Changhua Meng, Weiqiang Wang, and Liang Chen. 2024. Revisiting Modularity Maximization for Graph Clustering: A Contrastive Learning Perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*. Association for Computing Machinery, New York, NY, United States, 1968–1979. doi:10.1145/3637528.3671967
- [12] Atefeh Moradan, Andrew Draganov, Davide Mottin, and Ira Assent. 2023. UCoDe: Unified community detection with graph convolutional networks. *Machine Learning* 112, 12 (2023), 5057–5080. doi:10.1007/s10994-023-06402-0
- [13] Pascal Pons and Matthieu Latapy. 2005. Computing Communities in Large Networks Using Random Walks. In *Proceedings of the 20th International Symposium on Computer and Information Sciences (ISCIS 2005)*. Springer Berlin Heidelberg, 284–293. doi:10.1007/11569596_31
- [14] Stanislav Sobolevsky and Alexander Belyi. 2022. Graph neural network inspired algorithm for unsupervised network community detection. *Applied Network Science* 7, 1 (2022), 63. doi:10.1007/s41109-022-00500-z
- [15] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. 2014. General optimization technique for high-quality community detection in complex networks. *Physical Review E* 90, 1 (2014), 012811. doi:10.1103/PhysRevE.90.012811
- [16] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9, 1 (2019), 5233. doi:10.1038/s41598-019-41695-z
- [17] Anton Tsitsulin, John Palowitch, Bryan Perozzi, and Emmanuel Müller. 2023. Graph Clustering with Graph Neural Networks. *Journal of Machine Learning Research* 24, 127 (2023), 1–21. http://jmlr.org/papers/v24/20-998.html
- [18] Ken Wakita and Toshiyuki Tsurumi. 2007. Finding community structure in mega-scale social networks: [extended abstract]. In *Proceedings of the 16th International Conference on World Wide Web (WWW 2007)*. Association for Computing Machinery, 1275–1276. doi:10.1145/1242572.1242805
- [19] Hong Wang, Yinglong Zhang, Zhangqi Zhao, Zhicong Cai, Xuewen Xia, and Xing Xu. 2025. Simple yet effective heuristic community detection with graph convolution network. *Scientific Reports* 15, 1 (2025), 39249. doi:10.1038/s41598-025-22860-z
- [20] Tianqi Zhang, Yun Xiong, Jiawei Zhang, Yao Zhang, Yizhu Jiao, and Yangyong Zhu. 2020. CommDGI: Community Detection Oriented Deep Graph Infomax. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM 2020)*. Association for Computing Machinery, 1843–1852. doi:10.1145/3340531.3412042

A Research Methods

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009